

گزارش نهایی پروژه هوش مصنوعی - فاز دوم

مسیر ۱: عامل های هوشمند (Agent-Based AI)

پیاده سازی و بهبود عامل هوشمند در بازی Pacman با استفاده از الگوریتم Minimax

چکیده

در این پروژه، یک عامل هوشمند برای بازی Pacman با استفاده از الگوریتم Minimax طراحی و پیاده سازی شده است. هدف اصلی، ایجاد یک Agent است که بتواند ضمن جمع آوری تمام نقاط روی صفحه، از دسترس Ghost ها دور بماند. برای ارزیابی حالت ها، دو تابع سودمندی (Utility) مجزا برای Pacman (بیشینه ساز) و Ghost ها (کمینه ساز) طراحی گردید. مدل نهایی با اعمال بهبودهایی در توابع ارزیابی، همکاری هوشمندانه بین Ghost ها و جلوگیری از حرکت های سیکلی Pacman به عملکرد مطلوبی دست یافته است.

۱. توضیح کامل مدل نهایی

۱.۱. معماری کلی

مدل طراحی شده از نوع عامل مبتنی بر سودمندی (Utility-based Agent) است. Agent ها با نگاه به آینده (با عمق مشخص) بهترین حرکتی را انتخاب می کنند که بیشترین سود را برایشان به همراه داشته باشد. ترتیب حرکت به صورت Turn-Taking است:

1. Pacman حرکت می کند (Max).

2. Ghost 0 حرکت می کند (Min).

3. Ghost 1 حرکت می کند (Min).

این چرخه تا انتهای بازی ادامه می یابد.

۱.۲. الگوریتم اصلی Minimax :

الگوریتم Minimax به صورت کامل در فایل minmax.py پیاده سازی شده است. این الگوریتم تا عمق $MAX_DEPTH = 6$ درخت بازی را جستجو کرده و بهترین حرکت را برمی گرداند.

- عامل بیشینه ساز (Pacman) به دنبال بیشینه کردن مقدار تابع $utility()$ است.

- **عامل کمینه‌ساز (Ghost):** به دنبال کمینه‌کردن مقدار تابع `utility_ghost()` است (که در واقع معادل بیشینه‌کردن شانس رسیدن به Pacman است)

۱.۳. توابع سودمندی (Utility Functions)

۱.۳.۱. تابع سودمندی Pacman (utility)

این تابع وضعیت را از دید Pacman ارزیابی می‌کند. مؤلفه‌های اصلی آن عبارتند از:

- **شرط پایانی:** اگر Pacman خورده شود $-\infty$ و اگر تمام غذاها خورده شود $+\infty$.
- **امتیاز (Score):** امتیاز مستقیم بازی به عنوان پایه در نظر گرفته می‌شود.
- **اجتناب از روح (Ghost Avoidance):** بر اساس فاصله منتهی تا نزدیک‌ترین روح، جریمه شدید اعمال می‌شود. نزدیکی به روح (فاصله ۱ و ۲) به شدت منجر به کاهش سودمندی می‌شود.
- **جذب غذا (Food Attraction):** به نزدیک‌ترین غذا پاداش داده می‌شود. همچنین جریمه‌ای برای تعداد غذاهای باقی‌مانده در نظر گرفته می‌شود تا Agent تشویق به پاکسازی نقشه شود.

۱.۳.۲. تابع سودمندی Ghost (utility_ghost)

این تابع وضعیت را از دید یک Ghost ارزیابی کرده و برای الگوریتم Minimax منفی می‌شود (زیرا Ghost کمینه‌ساز است). مؤلفه‌های اصلی:

- **شرط پایانی:** اگر Ghost به Pacman برسد $+\infty$ (که در خروجی نهایی $-\infty$ می‌شود).
- **فاصله تا Pacman:** هرچه Ghost به Pacman نزدیک‌تر باشد، سودمندی پایه بالاتر است.
- **مسیریابی BFS:** در فواصل دور، از الگوریتم `bfs_to_pacman` برای یافتن کوتاه‌ترین مسیر استفاده می‌شود تا Ghost حرکتی کورکورانه انجام ندهد.
- **همکاری Ghost ها (Cooperation Bonus):** یک نوآوری در این پروژه است. اگر هر دو Ghost همزمان به Pacman نزدیک باشند و در دو سمت مختلف قرار گیرند، پاداش همکاری قابل توجهی دریافت می‌کنند. برعکس، اگر بیش از حد به هم نزدیک باشند، جریمه می‌شوند.

۲. نحوه آموزش

در پروژه‌های عامل‌محور (Agent-Based) مانند این پروژه، مفهوم "آموزش" به معنای بهینه‌سازی تابع سودمندی و پارامترهای جستجو است. مدل ما نیازی به "Training Loop" سنتی (مانند شبکه‌های عصبی) ندارد، بلکه با آزمایش پلن‌های مختلف حرکتی و تنظیم وزن‌های تابع Utility بهینه می‌شود.

فرآیند بهینه‌سازی:

1. تنظیم دستی وزن‌ها: وزن‌های مربوط به جریمه Ghost (اعداد ۸۰۰، ۴۰۰، ۱۵۰) و پاداش غذا (۳۰۰) بر اساس رفتار Agent در سناریوهای مختلف تنظیم شد.
2. تنظیم عمق جستجو: مقدار IMAX_DEPTH از ۲ تا ۸ آزمایش شد و در نهایت مقدار ۶ به دلیل تعادل بین سرعت و هوشمندی انتخاب گردید.
3. بهینه‌سازی همکاری Ghost‌ها: ضریب cooperation_bonus به صورت تجربی تنظیم شد تا Ghost‌ها یاد بگیرند به جای دنبال کردن تک‌تک، Pacman را محاصره کنند.

۴. تحلیل:

- افزایش عمق: منجر به برنامه‌ریزی بلندمدت‌تر شد.
- افزودن BFS: مشکل سرگردانی Ghost‌ها در فاصله دور را حل کرد.
- افزودن Cooperation Bonus: منجر به محاصره هوشمندانه Pacman و کاهش فضای حرکتی او شد.

۵. تحلیل خطا (Error Analysis)

۵.۱. خطای حرکت‌های سیکلی (Cyclic Movement)

مشکل: مشاهده شد که Pacman گاهی بین دو نقطه در حال رفت و برگشت بی‌هدف است.
راه حل: در فایل main.py، با بررسی تاریخچه حرکات (PACMAN_MOVES)، از انتخاب حرکتی که منجر به سیکل سه حرکتی می‌شود، جلوگیری شد. این یک **Heuristic** قوی برای جلوگیری از تکرار حرکات بی‌هوده است.

۵.۲. خطای برخورد با Obstacle

مشکل: در برخی موارد، Agent سعی در حرکت به سمت دیوار داشت که منجر به ایستادن و اتلاف وقت می‌شد.
راه حل: تابع is_valid_position پیش از تولید Child State، صحت موقعیت جدید را بررسی می‌کند. اگر موقعیت نامعتبر باشد، Agent در جای خود باقی می‌ماند.

۵.۳. خطای همپوشانی Ghost‌ها

مشکل Ghost: ها گاهی دقیقاً روی یک خانه می ایستادند.

راه حل: با اعمال جریمه `ghost_to_ghost_dist` در تابع `ghost.utility_ghost` ها یاد گرفتند که از یکدیگر فاصله بگیرند تا محدوده تحت پوشش بیشتری داشته باشند.

۶. معماری مدل و دیگرام

۶.۱. معماری ماژولار پروژه

ساختار پروژه به صورت کاملاً ماژولار و مطابق با استانداردهای پروژه‌های صنعتی طراحی شده است:

```
text
src/
|
# game_state.py — منطق اصلی بازی و توابع Utility
# minmax.py — پیاده‌سازی الگوریتم Minimax
# utils.py — توابع کمکی (BFS)، یافتن همسایه، تولید فرزند
# main.py — حلقه اصلی بازی و اجرای دمو
```

۷. اسکرین‌شات از دمو (نمونه اجرا)

۸. نتیجه‌گیری

در این پروژه، یک سامانه هوشمند کامل برای بازی Pacman با استفاده از الگوریتم Minimax طراحی و پیاده‌سازی گردید. دستاوردهای اصلی این پروژه عبارتند از:

- ✓ پیاده‌سازی کامل الگوریتم Minimax با قابلیت توسعه برای تعداد Agent های مختلف.
- ✓ طراحی توابع Utility کارآمد: با در نظر گرفتن همزمان پارامترهای فاصله، امتیاز، خطر و پاداش.
- ✓ ایجاد همکاری هوشمندانه بین Ghost ها: از طریق اعمال پاداش‌های همکاری و جریمه‌های هم‌پوشانی.
- ✓ بهینه‌سازی حرکتی: جلوگیری از سیکل‌های تکراری با استفاده از حافظه حرکات.
- ✓ معماری ماژولار و تمیز: جدا کردن منطق بازی، الگوریتم جستجو، توابع کمکی و حلقه اصلی از یکدیگر.

نقاط قوت:

- عامل Pacman در اکثر نقشه‌های آزمایشی موفق به پیروزی شد.
- Ghostها برخلاف مدل پایه، رفتاری هدفمند و محاصره‌کننده از خود نشان دادند.
- زمان پاسخ‌دهی الگوریتم با عمق ۶ برای اجرای بیدرنگ (Real-time) مناسب است.

کارهای آینده (پیشنهادی):

- پیاده‌سازی الگوریتم **Alpha-Beta Pruning** برای افزایش عمق جستجو بدون کاهش سرعت.
- اضافه کردن فاکتور ترس (**Fear**) برای Pacman به صورت پویا) هرچه Ghost نزدیک‌تر باشد، ضریب اجتناب افزایش یابد.
- پیاده‌سازی دموی گرافیکی با **Pygame** برای نمایش بصری جذاب‌تر.